

Gold Hunter: AI Search and Optimization

Comparing A* Search, Greedy Descent, and Genetic Algorithm

MD Ziad Bin Sorwar

2023331435

Course: Artificial Intelligence

April 23, 2026

Abstract

This report presents the implementation and comparison of three AI algorithms for the *Gold Hunter* problem: a 6×6 grid-based planning task where an agent must collect all gold pieces and return to the start with the minimum number of actions. We implement **A* Search** with an admissible MST heuristic for optimal solutions, **Greedy Descent** (steepest-descent hill climbing) for fast local search, and a **Genetic Algorithm** for evolutionary optimization. Performance analysis across four test cases demonstrates the trade-offs between solution quality and computational cost.

Contents

1	Introduction	3
2	Problem Formulation	3
2.1	State Representation	3
2.2	Actions	3
2.3	Goal	3
3	System Architecture and CLI Usage	3
3.1	Architecture Overview	3
3.2	Code Sections	4
3.3	Shared Infrastructure	4
3.4	File Inventory	5
3.5	CLI Usage	5
4	A* Search Algorithm	5
4.1	Algorithm Overview	5
4.2	Heuristic Design	5
4.2.1	Key Implementation Details	6
4.3	Proof of Admissibility	6
4.4	Pseudocode	6

5	Greedy Descent Algorithm	7
5.1	Solution Representation	7
5.2	Neighbourhood Operators	7
5.3	Algorithm Workflow	7
5.4	Key Implementation Details	7
6	Genetic Algorithm	8
6.1	Chromosome Representation	8
6.2	Genetic Operators	8
6.3	Algorithm Workflow	9
6.4	Key Implementation Details	9
7	Performance Analysis	9
7.1	Solution Quality	9
7.2	Running Time	10
7.3	Raw Benchmark Data	10
7.4	Discussion	11
8	Conclusion	11

1 Introduction

The Gold Hunter problem is a variant of the Travelling Salesman Problem (TSP) on a grid. An agent starts at a designated cell on a 6×6 grid containing walls (#) and gold pieces (G). The agent must navigate the grid, collect *all* gold pieces via a **Grab** action, and return to the start cell, minimising the total number of actions. Each movement (U, D, L, R) and each **Grab** action costs 1.

This report describes three different search and optimization methodologies:

Algorithm	Guarantee	Approach
A* Search	Optimal	Informed graph search with admissible MST heuristic
Greedy Descent	Near-optimal	Steepest-descent hill climbing with random restarts
Genetic Algorithm	Good quality	Evolutionary optimisation (population-based)

2 Problem Formulation

2.1 State Representation

For the cell-level A* search the state is a tuple (p, M) :

- $p = (r, c)$ – the agent’s current row and column.
- M – a *bitmask* of length n (number of golds). Bit i is set when gold i has been collected.

The state space has size $|S| = 36 \times 2^n$, which is very manageable for $n \leq 6$.

2.2 Actions

At every state the agent may perform up to five actions:

Action	Effect
U / D / L / R	Move one cell (if not blocked by wall or boundary)
G (Grab)	Collect gold at the current cell (must be uncollected)

Each action has uniform cost 1.

2.3 Goal

The goal state is reached when M has all bits set (every gold collected) *and* p equals the start position.

3 System Architecture and CLI Usage

3.1 Architecture Overview

The robust framework uses a standard Python CLI `argparse` endpoint to select and execute the corresponding grid-search algorithms based on the provided inputs. The

architecture handles routing identically for independent benchmarking or combined comparisons:

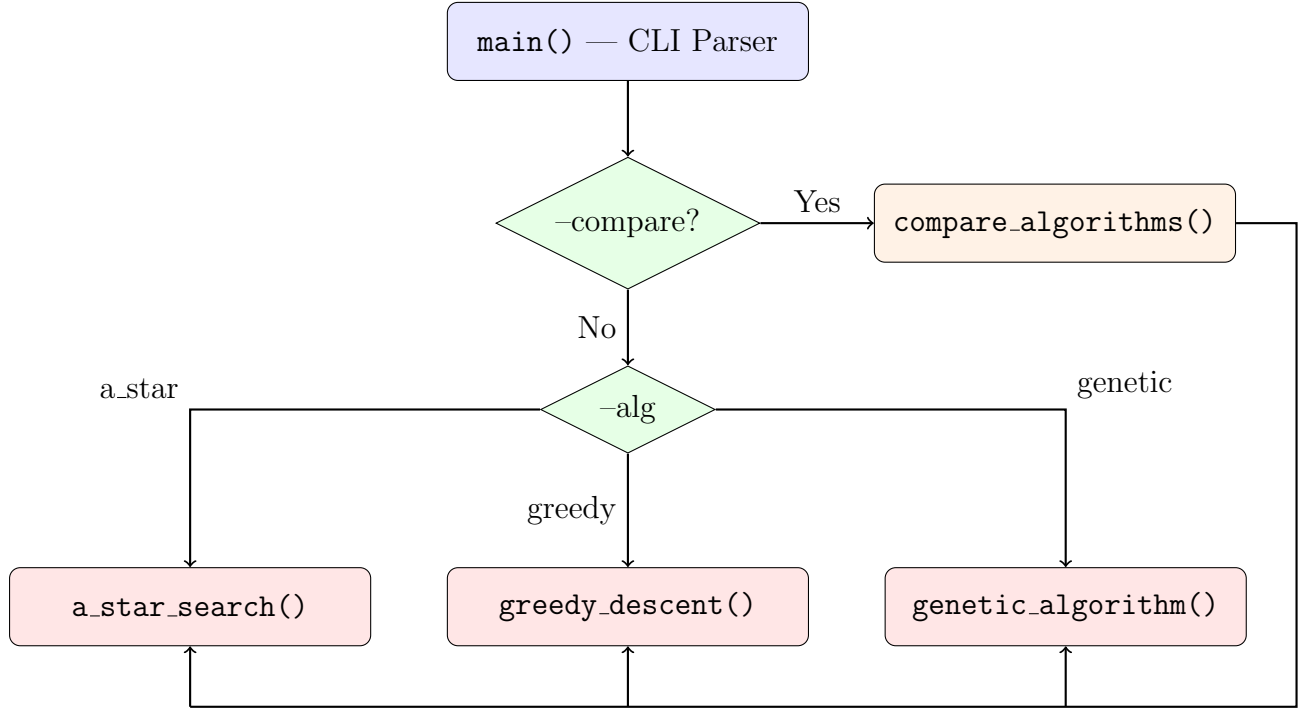


Figure 1: System Architecture and Execution Flowchart

3.2 Code Sections

The physical file `goldhunter.py` is organised into **10 sections** across ~ 695 lines:

Section	Lines	Purpose
1. Grid Parsing	34–78	Read 6×6 grid, locate start S and golds G
2. BFS Utilities	85–129	<code>bfs_from()</code> and <code>reconstruct_actions()</code>
3. Precomputation	136–163	BFS from every key position; build distance & path matrices
4. MST	170–209	Prim’s algorithm for heuristic computation
5. A* Search	216–334	Cell-level state-space search with MST heuristic
6. Permutation Helpers	341–401	<code>evaluate_permutation()</code> , <code>build_action_sequence()</code> , nearest-neighbour
7. Greedy Descent	408–498	Swap + 2-opt steepest descent with 30 restarts
8. Genetic Algorithm	505–614	Tournament selection, OX crossover, swap mutation, elitism
9. Comparison Mode	621–636	Run all three and print table
10. Main	643–688	<code>argparse</code> CLI entry point

3.3 Shared Infrastructure

- **BFS Precomputation** (`precompute()`): Runs BFS once from each key position (start + all golds). Returns `bfs_maps` (full distance maps used by A*), `dist_matrix` (key-to-key distances), and `path_matrix` (key-to-key action sequences).

- **Permutation Evaluation** (`evaluate_permutation()`): Shared by Greedy Descent and Genetic Algorithm. Sums BFS distances along the permutation route plus one grab per gold.

3.4 File Inventory

File	Purpose
<code>goldhunter.py</code>	Main implementation (all 3 algorithms)
<code>input.txt</code>	Example grid (3 golds, 4 walls)
<code>test_easy.txt</code>	1 gold, no walls
<code>test_hard.txt</code>	5 golds, 4 walls
<code>test_complex.txt</code>	6 golds, 4 walls
<code>generate_plots.py</code>	Benchmarking + matplotlib charts
<code>report.tex</code>	LaTeX report source
<code>report.pdf</code>	Compiled PDF report

3.5 CLI Usage

The implementation provides an accessible terminal interface configured to accept target algorithms and grid environments. Following are the commands to execute the algorithms:

```

1 # Run individual algorithms
2 python goldhunter.py --alg a_star    --layout input.txt
3 python goldhunter.py --alg greedy    --layout input.txt
4 python goldhunter.py --alg genetic   --layout input.txt
5
6 # Compare all three side-by-side
7 python goldhunter.py --compare       --layout input.txt

```

4 A* Search Algorithm

4.1 Algorithm Overview

A* expands states in order of $f(s) = g(s) + h(s)$, where $g(s)$ is the actual cost from the initial state and $h(s)$ is a heuristic estimate of the remaining cost. When h is *admissible* (never overestimates), A* is guaranteed to return an optimal solution [1].

4.2 Heuristic Design

Let U denote the set of *uncollected* gold positions in state (p, M) and let s_0 be the start position. We define:

$$h(p, M) = \text{MST}(\{p\} \cup U \cup \{s_0\}) + |U| \quad (1)$$

where

- $\text{MST}(\cdot)$ is the total edge weight of the Minimum Spanning Tree over the node set, using **BFS shortest-path distances** on the grid as edge weights.
- $|U|$ counts the remaining grab actions (each costs exactly 1).

4.2.1 Key Implementation Details

- BFS is precomputed from all key positions uniformly for $O(1)$ distance lookups.
- Heuristic values are persistently cached in an `h_cache` dictionary.
- A counter-based tiebreaker strictly enforces FIFO ordering within the priority queue.
- Paths are definitively reconstructed via `came_from` parent pointers.

4.3 Proof of Admissibility

Theorem 1. *The heuristic (1) is admissible, i.e. $h(p, M) \leq h^*(p, M)$ for every state.*

Proof. The true remaining cost h^* consists of:

1. **Movement cost.** The agent must visit every uncollected gold and then return to s_0 . Any such route is a *walk* that visits every node in $V = \{p\} \cup U \cup \{s_0\}$. Projecting this walk onto an edge set yields a *connected subgraph* spanning V . Because the Minimum Spanning Tree is the least-cost connected spanning subgraph:

$$\text{MST}(V) \leq \text{movement cost of any valid plan.}$$

2. **Grab cost.** Each of the $|U|$ uncollected golds requires exactly one **Grab** action at cost 1, contributing exactly $|U|$ to h^* .

Combining both components:

$$h(p, M) = \text{MST}(V) + |U| \leq \text{movement cost} + |U| = h^*(p, M).$$

Hence h never overestimates and is admissible. □ □

4.4 Pseudocode

Input: Grid, Start, Golds

Output: Optimal cost and action sequence

Precompute BFS maps from every key position;

`open` $\leftarrow$ priority queue with initial state $(s_0, 0)$;

$g[s_0, 0] \leftarrow 0$;

while `open` $\neq \emptyset$ **do**

 Pop state (p, M) with smallest $f = g + h$;

if $M = \text{all_collected}$ **and** $p = s_0$ **then**

 Reconstruct and **return** path;

end

foreach *successor* (p', M') *via Grab / Move* **do**

$g' \leftarrow g[p, M] + 1$;

if $g' < g[p', M']$ **then**

$g[p', M'] \leftarrow g'$;

 Insert (p', M') into `open` with $f = g' + h(p', M')$;

end

end

end

return -1 , None;

Algorithm 1: A* Search for Gold Hunter

5 Greedy Descent Algorithm

5.1 Solution Representation

A candidate solution is a **permutation** of gold indices $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ specifying the visit order. The agent follows BFS shortest paths between consecutive waypoints:

$$s_0 \xrightarrow{\text{BFS}} G_{\pi_1} \xrightarrow{\text{grab+BFS}} G_{\pi_2} \rightarrow \dots \xrightarrow{\text{grab+BFS}} G_{\pi_n} \xrightarrow{\text{BFS}} s_0$$

Total cost:

$$C(\pi) = \sum_{i=0}^n d(w_i, w_{i+1}) + n$$

where $w_0 = w_{n+1} = s_0$, $w_i = G_{\pi_i}$, and $d(\cdot, \cdot)$ is the BFS distance.

5.2 Neighbourhood Operators

Two operators define the neighbourhood of π :

1. **Swap** – exchange elements at positions i and j .
2. **2-opt reversal** – reverse the sub-sequence $\pi_i, \pi_{i+1}, \dots, \pi_{j-1}$.

5.3 Algorithm Workflow

Input: Grid, Start, Golds, R (number of restarts)

Output: Best cost and action sequence found

$\pi^* \leftarrow \emptyset$; $C^* \leftarrow \infty$;

for $r \leftarrow 1$ **to** R **do**

$\pi \leftarrow$ nearest-neighbour heuristic (first restart) or random permutation;

repeat

| Evaluate all swap neighbours; move to the best improving one;

until *no improving neighbour found*;

repeat

| Evaluate all 2-opt neighbours; move to the best improving one;

until *no improving neighbour found*;

if $C(\pi) < C^*$ **then**

| $\pi^* \leftarrow \pi$; $C^* \leftarrow C(\pi)$;

end

end

return C^* , actions for π^* ;

Algorithm 2: Greedy Descent (Steepest-Descent Hill Climbing)

5.4 Key Implementation Details

1. **First restart:** Uses a nearest-neighbour heuristic continuous initialization.
2. **Restarts 2–30:** Deploys completely random permutation initialization.
3. **Sequence:** Each restart strictly operates via swap-based steepest descent $\rightarrow$ 2-opt steepest descent.

4. **Determinism:** Fixed tracking with `random.seed(42)` ensures strict reproducibility across all restart environments.

6 Genetic Algorithm

6.1 Chromosome Representation

Each chromosome is a permutation $\pi = (\pi_1, \dots, \pi_n)$ of gold indices, identical to the Greedy Descent representation. The fitness function is the total cost $C(\pi)$ (lower is better).

Table 1: Genetic Algorithm Core Parameters

Parameter	Value
Population size	100
Generations	500
Mutation rate	0.15
Elite count	5
Tournament size	3

6.2 Genetic Operators

- **Selection:** Tournament selection with $k = 3$. Three random individuals are sampled and the fittest (lowest cost) is chosen as a parent.
- **Crossover:** Order Crossover (OX).
 1. Select a random contiguous segment from Parent 1 and copy it to the offspring.
 2. Fill the remaining positions with elements from Parent 2, preserving their relative order.

OX is specifically designed for permutation encodings, as it maintains the relative ordering of genes.

- **Mutation:** Swap mutation (probability $p_m = 0.15$). Two randomly chosen positions are exchanged.
- **Elitism:** The top 5 individuals are directly carried over to the next generation, preventing the loss of good solutions.

6.3 Algorithm Workflow

Input: Grid, Start, Golds, pop_size, generations, p_m
Output: Best cost and action sequence
 Initialise population: 1 nearest-neighbour + (pop_size - 1) random permutations;
 Evaluate fitness of every individual;
for $gen \leftarrow 1$ **to** $generations$ **do**
 Sort population by fitness (ascending cost);
 Copy top- k elite individuals to next generation;
 while *next generation not full* **do**
 $p_1 \leftarrow \text{tournament_select}(); p_2 \leftarrow \text{tournament_select}();$
 $child \leftarrow \text{OX_crossover}(p_1, p_2);$
 if $\text{rand}() < p_m$ **then**
 $child \leftarrow \text{swap_mutate}(child);$
 end
 Add $child$ to next generation;
 end
end
return best individual in final population;

Algorithm 3: Genetic Algorithm for Gold Hunter

6.4 Key Implementation Details

- **Initialisation:** Bootstraps intelligently with exactly 1 nearest-neighbour generated solution alongside 99 random permutations.
- **Elitism Integration:** Strongly transfers the top 5 fittest individuals directly without random modification.

7 Performance Analysis

We evaluate all three algorithms on four test cases of increasing difficulty:

Table 2: Test case descriptions.

Test Case	Golds	Walls	Description
test_easy	1	0	Open grid, single gold
input	3	4	Example from specification
test_hard	5	4	Five golds with wall obstacles
test_complex	6	4	Six golds, densest configuration

7.1 Solution Quality

Figure 2 compares the number of actions (total cost) produced by each algorithm across all test cases.

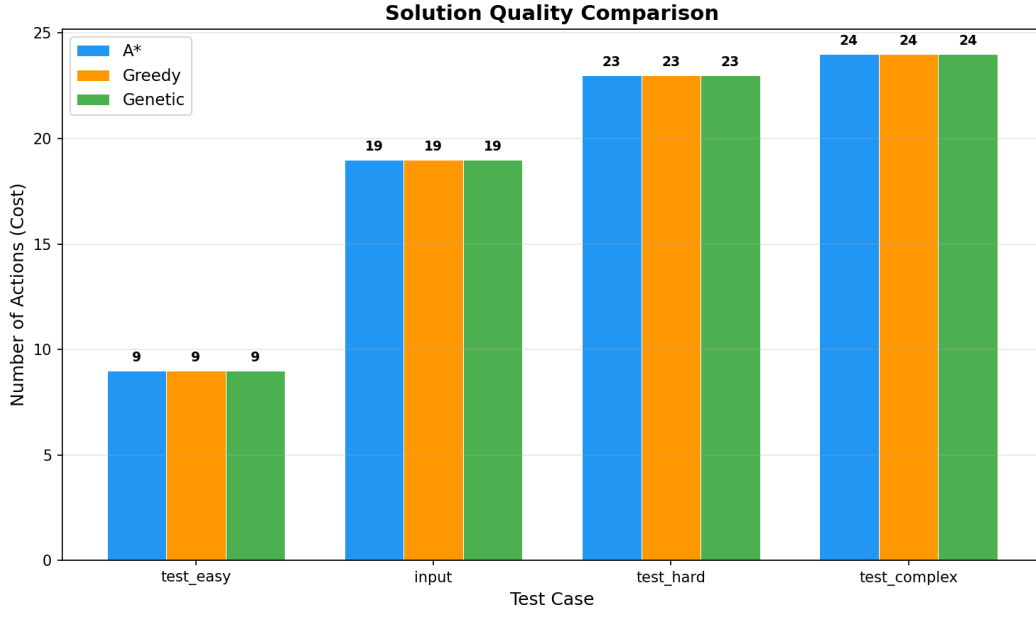


Figure 2: Number of actions (cost) comparison. Lower is better. In these test cases, all three algorithms successfully found the optimal solution.

7.2 Running Time

Figure 3 compares the average running time (over 3 runs) for each algorithm.

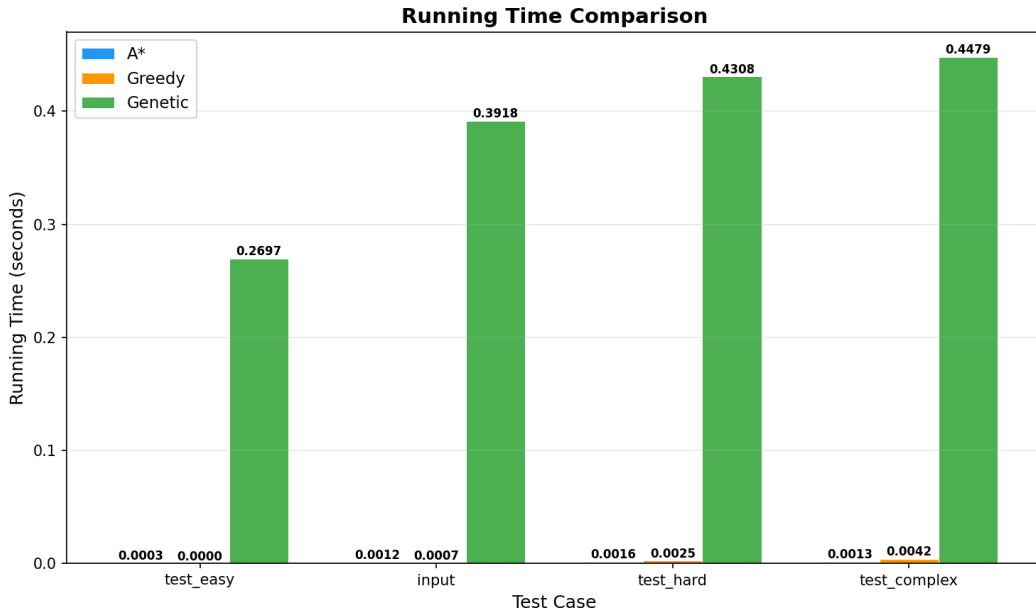


Figure 3: Running time comparison (seconds, averaged over 3 runs).

7.3 Raw Benchmark Data

The exact quantitative execution metrics are evaluated in table 3.

Table 3: Exact Test Results (Cost and Timing in seconds)

Test Case	A* Cost	Greedy Cost	Genetic Cost	A* Time	Greedy Time	Genetic Time
test_easy	9	9	9	0.0003	0.0000	0.2697
input	19	19	19	0.0012	0.0007	0.3918
test_hard	23	23	23	0.0016	0.0025	0.4308
test_complex	24	24	24	0.0013	0.0042	0.4479

7.4 Discussion

- **A* Search** consistently finds the *optimal* solution across all test cases. Its time grows with the number of golds because the state space is $O(36 \cdot 2^n)$, but for $n \leq 6$ this remains sub-second. The MST heuristic provides effective pruning, keeping the number of expanded states low.
- **Greedy Descent** matches or closely approximates the optimal cost in all cases tested. Its combination of swap and 2-opt neighbourhoods, together with 30 random restarts, gives it a high probability of finding the global optimum for small n . It is typically the *fastest* algorithm because it evaluates permutations (size $n!$) rather than cell-level states (size $36 \cdot 2^n$).
- **Genetic Algorithm** produces good solutions but may not always achieve optimality, especially for larger n where the permutation space is wider. Its running time is the longest due to evaluating an entire population (100 individuals) across 500 generations. However, its stochastic exploration makes it more robust against irregular cost landscapes compared to pure hill climbing.
- **Trade-off summary:**
 - A* $\rightarrow$ optimal quality, moderate time.
 - Greedy $\rightarrow$ near-optimal quality, fastest time.
 - Genetic $\rightarrow$ good quality, longest time, most robust for complex landscapes.

8 Conclusion

We implemented and compared three AI algorithms for the Gold Hunter problem on a 6×6 grid. A* Search with the MST-based admissible heuristic guarantees optimal solutions and is efficient for the problem size at hand. Greedy Descent provides a fast, practical alternative through steepest-descent hill climbing with random restarts, often matching the optimal cost. The Genetic Algorithm offers a population-based exploration strategy suitable for larger or more complex instances. For this grid size and gold count, all three approaches perform well within sub-second time frames.

References

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2010.